

Data Retrieval with get method

In Hibernate, the `get()` method is used to retrieve an entity from the database based on its primary key. It fetches the data and converts it into a Java object automatically. Here's a detailed explanation of how data retrieval works using Hibernate's `get()` method:

1. Entity Definition (Student Class)

First, we define the entity class that is mapped to the database table. In this case, the class `Student` is mapped to the `StudentTable` in the database.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id
    @Column(name="sid")
    private int id;

    @Column(name="sname")
    private String name;

    @Column(name="scity")
    private String city;

    public Student() {
        System.out.println("Zero param constructor");
    }
}
```

```

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
}

```

2. Data Flow During Retrieval

Here's how data retrieval happens step-by-step using get():

- **Step 1:** The get() method is called on the Session object. For example:

```
Student student = session.get(Student.class, 1);
```

- **Step 2:** Hibernate (ORM) takes care of the following:
 - It reads the **mapping details** from annotations (@Entity, @Table, @Column) to understand which table and columns to target.
 - It uses the configuration settings for the database connection.
- **Step 3:** Hibernate translates the get() method call into a SQL query using JDBC:

```
SELECT * FROM StudentTable WHERE sid=1;
```

- **Step 4: JDBC** executes the query on the database, retrieving the record.
- **Step 5:** Hibernate processes the result set. It finds the matching fields from the result set (sid, sname, scity) and maps them to the corresponding fields in the Student class.
- **Step 6:** Hibernate automatically creates an instance of the Student class and populates the fields with the data from the result set.
- **Step 7:** Hibernate calls the **zero-parameter constructor** (Student()) of the entity class and sets the values using setter methods (setId(), setName(), setCity()).

3. ResultSet to Object Mapping

Internally, Hibernate uses **ResultSet** from JDBC to get the data from the database. For each record in the ResultSet:

- It **calls the constructor** of the Student class to create an object.
- It **sets the values** of each field (id, name, city) using reflection and setter methods.

4. Main Class to Demonstrate Retrieval

Here's a simple main class that demonstrates how to retrieve data using Hibernate's get() method.

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class Main {
    public static void main(String[] args) {
        // Configure Hibernate
        SessionFactory factory = new
Configuration().configure("hibernate.cfg.xml")
        .addAnnotatedClass(Student.class).buildSessionFactory();

        // Open session
```

```

Session session = factory.openSession();

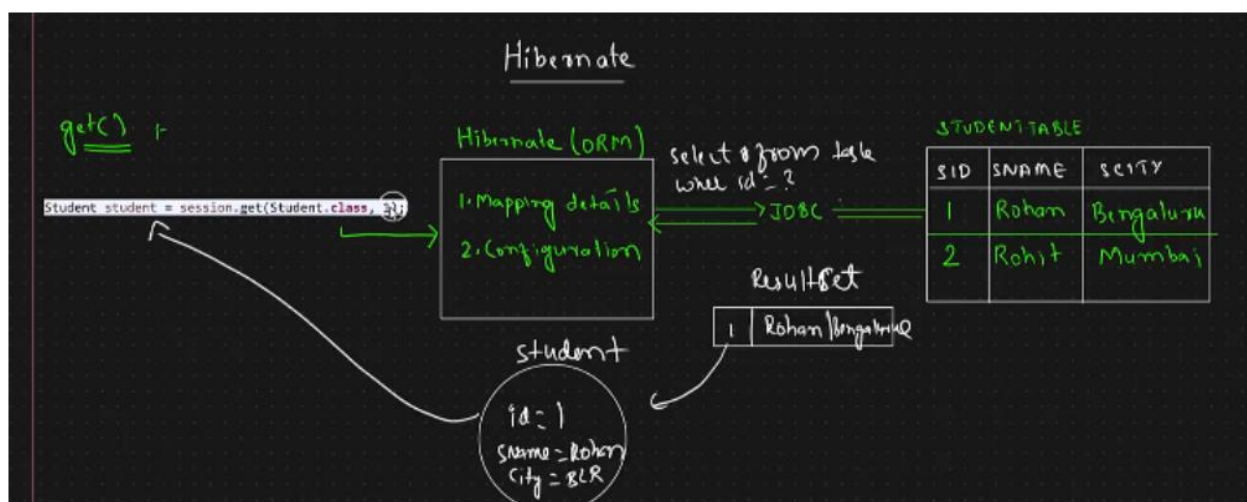
try {

    // Retrieve student with ID 1
    Student student = session.get(Student.class, 1);

    // Display the student details
    System.out.println("ID: " + student.getId());
    System.out.println("Name: " + student.getName());
    System.out.println("City: " + student.getCity());

} finally {
    // Close session
    session.close();
    factory.close();
}
}

```



5. How ORM Works Behind the Scenes

- Hibernate simplifies the database interaction by mapping Java objects (entities) to database tables.
- The `get()` method abstracts away the complexity of SQL queries and JDBC code.
- **Automatic Object Creation:** Hibernate takes care of object creation from database records. You don't have to manually write code to parse the `ResultSet`.
- **Zero-Parameter Constructor:** Hibernate requires a no-arg constructor to create instances of the entity class.

6. Advantages of `get()` Method

- **Simple API:** The `get()` method is simple to use and returns null if no entity is found.
- **Eager Loading:** The `get()` method fetches the object immediately and returns a fully initialized entity.
- **Convenience:** Hibernate handles mapping and data retrieval without manual SQL queries.